# ELEC 328 Project Report
# Assn 2 – Enchanted Flashlight
# Jonathan Hinojosa – S01460164 – jh218

## 1. Concept

For this project, we were tasked with implementing an “Enchanted Flashlight” using Unity Shader Graphs. The enchanted flashlight is meant to show hidden textures beneath a default one. This idea of hiding a world beneath the surface is tied, also, to ocean life. Thus, I decided to create a Unity scene which is meant to evoke being in an aquarium, getting to see what lies beneath the surface of the sea.

## 2. Technical overview

The project I created has two primary components which are relevant to this assignment: The Flashlight Object, and EnchantedFlashlightLit.shadergraph.

**The Flashlight Object:**

In my implementation of a grabbable flashlight, I used The Polynest’s publicly-available Tool_Flashlight_1 prefab object and an XR Grab Interactable to create the Flashlight object within my scene, titled EnchantedFlashlight. This Flashlight Object uses the inherent qualities of the XR Grab Interactable to enable a player to grab the flashlight with the grab trigger. By using a collider within the Flashlight object to detect when the player’s controller is touching the Flashlight and the XR Grab Interactable’s capability to allow for the Flashlight to follow the player’s controller after the grab trigger is pressed while the controller is colliding with the Flashlight, the player is enabled to grab the Flashlight and move it and its child objects.

**EnchantedFlashlightLit.shadergraph:**

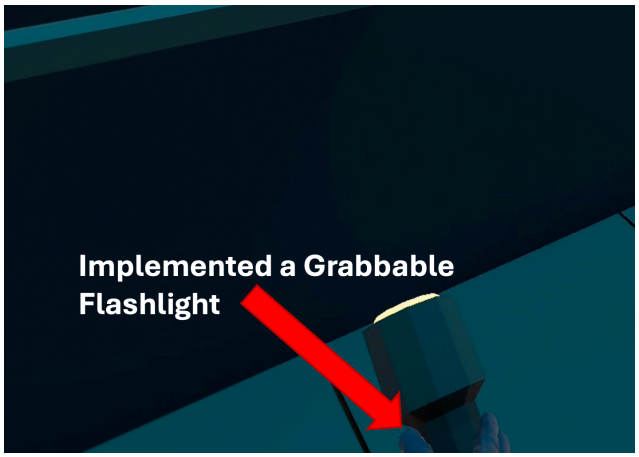
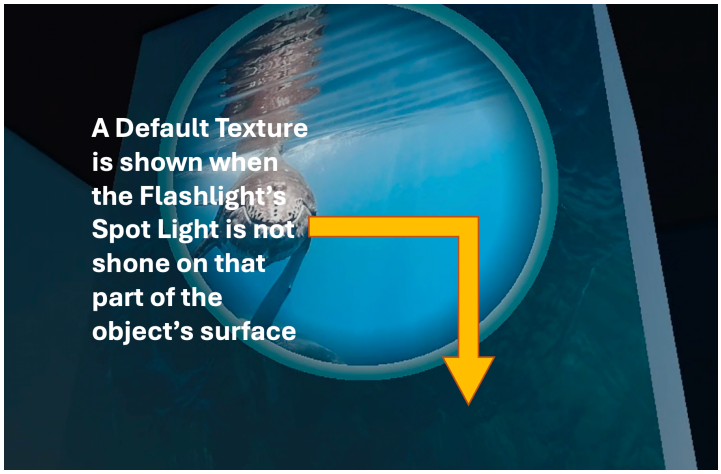
This Unity Shader Graph enables the Flashlight object and its children to become “enchanted.” Using the getspotlightinfo.cs script, this shader graph can receive information about the Flashlight object’s child Spot Lights (Spot_Whale, Spot_Penguin, Spot_Reef). This information consists of CustomSpotDirection, getting the Vector3 direction of a connected spot light, CustomSpotPosition, getting the Vector3 position of a connected spot light, and CustomSpotAngleCos, getting the Float value of the cosine of the conical light angle of the connected spot light.

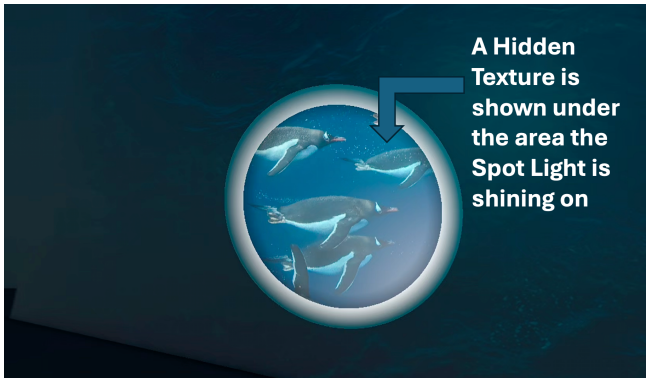
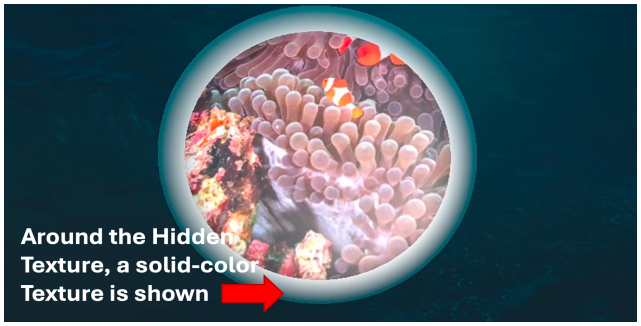
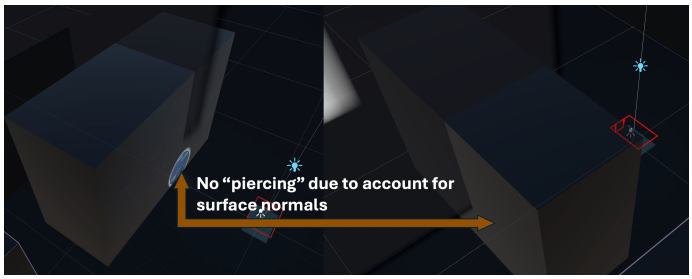
The shader graph can then use this information in the following order to determine whether or not a fragment is within the spot light’s cone of light, is facing the spot light, and if it should be the Base-, Illuminated-, or RingTexture. First, the normalized vector between CustomSpotPosition and the position of the fragment is retrieved. Second, in parallel, this normalized vector is used both to check whether or not the fragment is within the cone of the spot light by computing the dot product of this normalized vector and CustomSpotDirection and whether or not the fragment is facing the spot light by comparing the negative of the normalized vector to the normal vector of the fragment. Third, the results from the dot product are passed into two Step nodes to create an inner mask and an outer mask, to create the outer ring of RingTexture around the hidden texture. The outer mask checks the dot product against the CustomSpotAngleCos, while the inner mask checks the dot product against the

CustomSpotAngleCos added to a RingThickness float property. These different calculations enable the script to find fragments just at the edge of where the spot light is shining. The binary masks are ultimately created using step nodes for both of these processes. Fourth, both of these step node results are multiplied by the result of the fragment facing check to ensure the effect does not appear on the back of the object. Finally, cascading Lerp nodes use these multiplied masks to blend the textures. The first Lerp uses the outer mask to change between the BaseTexture and the RingTexture, and the second Lerp uses the inner mask to change between the output of that first Lerp and the IlluminatedTexture.

## 3. Evidence of requirements

[Use this table to point to the exact evidence for each requirement. Include video timestamps and screenshot labels.]

| Requirement | Where it is demonstrated | Screenshot |
|---|---|---|
| Grabbable Flashlight | ShowingGrabbableFlashlight.png |   |
| Surfaces Outside Flashlight Spot Light display a default texture | ShowingDefaultTexture.png |   |

| | | |
|---|---|---|
| Surfaces Inside Flashlight Spot Light display a hidden texture | ShowingHiddenTexture.png |  |
| Boundary of the Flashlight's Spot Light outlined with solid color | ShowingSolidColor.png |  |
| Ensuring the texture change only occurs on the side the spot light is shining on | ShowingHiddenSide_FrontandBack.png |  |

## 5. Assets, data, and citations

*List every external asset, dataset, model, texture, sound, library, or reference used. State whether it was created by you, provided by the course, or obtained elsewhere. Provide a source link or formal citation where appropriate.*

| Item | Source / citation | How it was used |
|---|---|---|
| Roost Rails | Course-Provided Asset | Used to mimic the rails of the Reality Roost. Also, used for Reality Roost compatibility. |

| | | |
|---|---|---|
| Gentoo penguins under the surface, Antarctic Peninsula | Creator: Lars von Ritter Zahony<br>Image URL: https://d2zmi9say0r1yj.cloudfront.net/OceanImageBank_LarsvonRitterZahony_17.jpg<br>Hosted at: https://oceanimagebank.theoceanagency.org/ | Used as the Hidden Texture for Penguin_Cube. |
| Coral reef, Raja Ampat, Indonesia | Creator: Dani Escayola<br>Image URL: https://d2zmi9say0r1yj.cloudfront.net/OceanImageBank_DaniEscayola_20.jpg<br>Hosted at: https://oceanimagebank.theoceanagency.org/ | Used as the Hidden Texture for Reef_Cube. |
| Humpback whales, Reunion Island | Creator: François Baelen<br>Image URL: https://d2zmi9say0r1yj.cloudfront.net/OceanImageBank_FrancoisBaelen_15.jpg<br>Hosted at: https://oceanimagebank.theoceanagency.org/ | Used as the Hidden Texture for Whale_Cube |
| FREE Demo: Low Poly Post-Apo Survival Tools (4 Assets) | Creator: The PolynestLab<br>Link: https://assetstore.unity.com/packages/3d/props/tools/free-demo-low-poly-post-apo-survival-tools-4-assets-331744 | Used this asset pack's Tool_Flashlight_1 Prefab as my flashlight model. |

## 6. AI-use disclosure

*If you used generative AI during project development, disclose it below. If you did not use AI, write: No generative AI was used in project development.*

| Tool | Used for | What was incorporated, changed, or rejected |
|---|---|---|
| Google Gemini | Shadergraph Node Understanding, C# syntax and Unity Library | When using nodes like Lerp, and Step, it helped me understand the inner-workings of these nodes so I knew what to input/output and why. Also, helped me in creating my getspotlightinfo.cs and oscillator.cs scripts as I am still not totally familiar with the C# language and the Unity libraries. |

## 7. Reflection

All in all, I think that this project went well for me. The mathematical concepts were easy to understand coming from understanding linear algebra. This enabled me to quickly create my shader graph using nodes that mimicked the mathematical processes I needed to calculate whether a fragment should be showing the hidden texture, the ring texture, or the default texture.

Of course, there were hiccups. When moving to unity, there were some things I had to look up and ask for help on; specifically, when I was looking for a method to get my texture masks onto the base texture in my shader graph, I needed to consult Gemini to find the correct node for my implementation. Also, this week I was unfortunately not able to give this project the time it really needed due to preparing for exams, and job interivews. This meant that I spent multiple nights this week staying up until 6-7:00 am doing work. In a way, I think my ability to complete this project suffered due to that.

Nevertheless, I was able to stretch creatively with this project in a way I felt I was not in the last one. I'm very satisfied with how the Unity experience turned out in the end; I only hope I have more time to create during the next project.